

1. Single Responsibility Principle (SRP)

Principle: Each class should have only one reason to change, meaning it should have only one job or responsibility.

✓ Strong SRP Implementation

The project demonstrates excellent adherence to SRP through its service-oriented architecture:

Service Layer Classes

Each service class has a single, well-defined responsibility:

- CustomerService.java (Line 1-500+)
 - Single Responsibility: User authentication, registration, and account management
 - Evidence: Handles only customer-related operations like login, registration, password management, and account security
 - Methods Focus: authenticateCustomer(), registerCustomer(), isValidPassword(), updateCustomer()
- GadgetService.java
 - Single Responsibility: Device validation and creation logic
 - Evidence: Contains only device-type validation, model validation, and gadget creation
 - Methods Focus: isValidDeviceType(), isValidModel(), createGadget(), getValidRooms()
- TimerService.java
 - Single Responsibility: Device scheduling and timer management
 - Evidence: Handles only timer-related operations and background task execution
 - Methods Focus: scheduleDeviceTimer(), cancelTimer(), checkScheduledTimers()
- AlertService.java (Lines 9-327)
 - Single Responsibility: Alert management and notification system
 - Evidence: Manages only alert creation, monitoring, and triggering
 - Methods Focus: createTimeBasedAlert(), createEnergyUsageAlert(), checkTimeBasedAlerts()
- EnergyManagementService.java (Lines 8-207)
 - Single Responsibility: Energy consumption tracking and cost calculation
 - Evidence: Handles only energy-related calculations and reporting
 - Methods Focus: generateEnergyReport(), calculateSlabBasedCost(), displayDeviceEnergyUsage()

Model Classes

- Customer.java: Represents user entity with user-specific data and operations
- Gadget.java: Represents IoT device entity with device-specific state and behavior
- CalendarEvent.java: Represents calendar events with event-specific data

Utility Classes

- SessionManager.java: Single responsibility for user session management
- DynamoDBConfig.java: Single responsibility for database configuration
- PasswordUtils.java: Single responsibility for password encryption utilities

SRP Benefits Observed:

1. Easy Testing: Each service can be tested independently
2. Maintainability: Changes to authentication logic don't affect device management
3. Code Reusability: Services can be reused across different parts of the application
4. Clear Architecture: Easy to understand what each class does

2. Open/Closed Principle (OCP)

Principle: Software entities should be open for extension but closed for modification.

✅ Good OCP Implementation

Alert System Inheritance (AlertService.java Lines 24-110)

The alert system demonstrates excellent OCP implementation through inheritance:

```
java
// Base class - closed for modification
public static class Alert {
    // Core alert functionality
}

// Extensions - open for extension
public static class TimeBasedAlert extends Alert {
    private LocalDateTime triggerTime;
    // Time-specific functionality
}

public static class EnergyUsageAlert extends Alert {
    private double energyThreshold;
    // Energy-specific functionality
}
```

Extension Points:

- New alert types can be added without modifying existing Alert class
- Each alert type extends base functionality without changing core logic
- Polymorphic behavior allows treating all alerts uniformly

Service-Based Architecture (SmartHomeService.java Lines 24-35)

The main service orchestrator demonstrates OCP:

```
java
public SmartHomeService() {
    this.customerService = new CustomerService();
    this.gadgetService = new GadgetService();
    this.energyService = new EnergyManagementService();
    // New services can be added without modifying existing code
}
```

Device Type Extensibility (GadgetService.java)

- Device validation system supports adding new device types
- Room types can be extended without modifying core validation logic

- Model validation follows extensible patterns

Areas for Further OCP Enhancement:

1. Interface-based Design: Could benefit from interfaces for services
2. Plugin Architecture: Could support runtime loading of new device types
3. Strategy Pattern: Could implement for energy calculation strategies

3. Liskov Substitution Principle (LSP)

Principle: Objects of a superclass should be replaceable with objects of its subclasses without breaking functionality.

Proper LSP Implementation

Alert Class Hierarchy (AlertService.java Lines 76-110)

The Alert inheritance hierarchy properly implements LSP:

```
java
// Base Alert class defines contract
public static class Alert {
    // Base functionality that all alerts must support
    public String getAlertId() { return alertId; }
    public boolean isActive() { return isActive; }
    // ... other base methods
}

// TimeBasedAlert is substitutable for Alert
public static class TimeBasedAlert extends Alert {
    // Extends without breaking base contract
    public TimeBasedAlert(...) {
        super(...); // Properly calls parent constructor
    }
    // Additional methods don't override base behavior
}

// EnergyUsageAlert is substitutable for Alert
public static class EnergyUsageAlert extends Alert {
    // Extends without breaking base contract
    public EnergyUsageAlert(...) {
        super(...); // Properly calls parent constructor
    }
}
```

LSP Compliance Evidence:

1. Polymorphic Usage (Lines 217-233):

```
java
for (Alert alert : alerts) {
    if (alert instanceof TimeBasedAlert) {
        TimeBasedAlert timeAlert = (TimeBasedAlert) alert;
        // TimeBasedAlert can be treated as Alert
    }
}
```

```
}

```

2. Substitutability: Any Alert reference can hold TimeBasedAlert or EnergyUsageAlert objects
3. Contract Preservation: Subclasses don't weaken preconditions or strengthen postconditions
4. Behavioral Consistency: All alert types maintain expected alert behavior

Proper Inheritance Design:

- Subclasses extend functionality without breaking parent class contracts
- Method overriding is done correctly where it exists
- No inappropriate strengthening of preconditions or weakening of postconditions

4. Interface Segregation Principle (ISP)

Principle: No code should be forced to depend on methods it does not use.

Mixed ISP Implementation

Since the codebase doesn't use interfaces extensively, ISP analysis focuses on class design and method grouping:

Positive ISP Aspects:

1. Focused Service Classes: Each service class provides only relevant methods to its clients
 - CustomerService: Only authentication and user management methods
 - GadgetService: Only device validation and creation methods
 - EnergyManagementService: Only energy calculation methods
2. Minimal Dependencies: Classes don't expose unnecessary functionality
 - SessionManager provides only session-related methods
 - AlertService provides only alert-related operations

Areas Needing ISP Attention:

1. SmartHomeService (SmartHomeService.java Lines 13-1541):
 - Issue: Large class with many methods serving different client needs
 - Impact: Clients using only device management still depend on energy management methods
 - Recommendation: Could be split into focused interfaces:

```
java
interface DeviceManager { / device methods / }
interface UserManager { / user methods / }
interface EnergyManager { / energy methods / }
```
2. Customer Model Class: Contains both user data and group management functionality
 - Could benefit from separating concerns into focused interfaces

ISP Improvement Recommendations:

1. Introduce service interfaces to segregate functionality
2. Create role-based interfaces (Admin, User, DeviceController)
3. Implement focused facades for different client types

5. Dependency Inversion Principle (DIP)

Principle: High-level modules should not depend on low-level modules. Both should depend on abstractions.

⚠️ Partial DIP Implementation

❌ DIP Violations:

1. Direct Service Dependencies (SmartHomeService.java Lines 24-35):

```
java
public SmartHomeService() {
    this.customerService = new CustomerService();           // Direct
    this.gadgetService = new GadgetService();               // Direct dependency
    this.energyService = new EnergyManagementService();     // Direct dependency
}
```

Issue: High-level SmartHomeService directly depends on concrete low-level services

2. Concrete Class Dependencies: Services directly instantiate other concrete services
3. No Abstraction Layer: Missing interfaces that could serve as abstractions

✅ Some DIP-Friendly Patterns:

1. **Singleton Pattern Usage:** Services accessed through static methods provide some abstraction

```
java
TimerService.getInstance()
AlertService.getInstance()
```

2. **Configuration Abstraction (DynamoDBConfig.java):**

- Provides abstraction over database connectivity
- Shields services from direct database implementation details

DIP Improvement Recommendations:

1. Introduce Service Interfaces:

```
java
public interface ICustomerService {
    Customer authenticateCustomer(String email, String password);
    boolean registerCustomer(String name, String email, String password);
}

public class SmartHomeService {
```

```

private final ICustomerService customerService;
private final IGadgetService gadgetService;
public SmartHomeService(ICustomerService customerService, IGadgetService gadgetService) {
    this.customerService = customerService;
    this.gadgetService = gadgetService;
}
}

```

2. Dependency Injection: Implement constructor or setter injection
3. Factory Pattern: Use factories to create service instances
4. IoC Container: Consider using Spring or similar framework

Summary and Recommendations

Strengths:

1.  Excellent SRP: Clear separation of concerns across all service classes
2.  Good OCP: Alert system and service architecture support extension
3.  Proper LSP: Alert inheritance hierarchy follows substitution principle
4.  Focused Classes: Most classes have cohesive responsibilities

Areas for Improvement:

1. ISP Enhancement: Introduce service interfaces to segregate functionality
2. DIP Implementation: Reduce concrete dependencies through abstractions
3. Interface Design: Add interfaces for major service contracts

Recommended Architectural Changes:

1. Create service interfaces for dependency inversion
2. Implement dependency injection pattern
3. Consider breaking down large service classes
4. Add abstraction layers for external dependencies

Overall SOLID Score: 7/10

The codebase demonstrates strong understanding and implementation of most SOLID principles, with particular excellence in SRP and good implementation of OCP and LSP. The main areas for improvement are ISP and DIP implementation through interface-based design.

File References

- SmartHomeService.java: Lines 13-1541 (Main service orchestrator)
- AlertService.java: Lines 9-327 (Alert system with inheritance)
- CustomerService.java: Lines 1-500+ (User management)
- GadgetService.java: Device validation and creation
- EnergyManagementService.java: Lines 8-207 (Energy calculations)
- TimerService.java: Device scheduling
- SessionManager.java: Session management
- DynamoDBConfig.java: Database configuration